

1. write a go progrma to declare variables of different data types (integer, floa, and string), assign values to them, and print those values.

```
package main
import "fmt"

func main() {
    var a int
    var b float32
    var c string

    a = 12
    b = 35.5
    c = "abc"

    fmt.Println("Integer a = ", a)
    fmt.Println("Float b = ", b)
    fmt.Println("String c = ", c)
}
```

2. Write a Go program to perform basic arithmetic operations (addition, subtractin, multilication, and division) between two numbers

```
package main
import "fmt"
func main() {
    var n1, n2, add, sub, mul, div int

    n1 = 20
    n2 = 10

    add = n1 + n2
    sub = n1 - n2
    mul = n1 * n2
    div = n1 / n2

    fmt.Println("n1 : ", n1)
    fmt.Println("n2 : ", n2)

    fmt.Println("Addition : ", add)
    fmt.Println("Subtraction : ", sub)
    fmt.Println("Multiplication : ", mul)
    fmt.Println("Division : ", div)
}
```

3. Write a G program to find the area of a rectangle. Take the length and width as inputs and display the area.

```
package main
import "fmt"
func main() {

    var length, width, area float32

    fmt.Print("Enter length : ")
    fmt.Scanf("%f", &length)

    fmt.Print("Enter width : ")
    fmt.Scanf("%f", &width)

    area = length * width

    fmt.Println("Area of Rectangle : ", area)
}
```

4. Write a Go program to calculate the perimeter of a square. Take the side length as input and display.

```
package main
import "fmt"
func main() {
    var length, perimeter float32

    fmt.Print("Enter length of side of square : ")
    fmt.Scanf("%f",&length)

    perimeter = 4 * length

    fmt.Println("The perimeter of square is : ", perimeter)
}
```

1. Write a Go program to calculate the area of a circle. Take the radius as input and display the area (use math.pi for the value of pi)

```
package main
import (
    "fmt"
    "math"
)
func main() {
    var radius, area float64

    fmt.Print("Enter radius : ")
    fmt.Scanf("%f",&radius)

    area = math.Pi * math.Pow(radius, 2)

    fmt.Println("Area of circle : ", area)
}
```

2. Write a Go program to calculate the result of a simple mathematical expression (for example : $(x + y) * z$) where x, y, and z are input by the user.

```
package main
import "fmt"
func main() {

    var x, y, z int

    fmt.Print("Enter x : ")
    fmt.Scanf("%d",&x)

    fmt.Print("Enter y : ")
    fmt.Scanf("%d",&y)

    fmt.Print("Enter z : ")
    fmt.Scanf("%d",&z)

    fmt.Println("\n (x + y) * z = ", (x + y) * z)
}
```

3. Write a Go program to perform the square of a number. Take an integer input and display the result.

```
package main
import (
    "fmt"
    "math"
)
func main() {

    var n, sq float64

    fmt.Print("Enter a number : ")
    fmt.Scan("%f",&n)

    sq = math.Pow(n, 2)

    fmt.Println("The square of given number is : ", sq)
}
```

4. Write a Go program to calculate the volume of a cylinder. Take the radius and height as inputs and display the volume.

```
package main
import (
    "fmt"
    "math"
)
func main() {
    var radius, height, volume float64

    fmt.Print("Enter radius : ")
    fmt.Scanf("%f",&radius)

    fmt.Print("Enter height : ")
    fmt.Scanf("%f",&height)

    volume = math.Pi * math.Pow(radius, 2) * height

    fmt.Printf("The volume of cylinder is : %.2f", volume)
}
```

1. Write a go program that checks if a number is positive, negative, or zero.

```
package main
import "fmt"
func main() {

    var n int;

    fmt.Print("Enter a number : ");
    fmt.Scanf("%d", &n);

    if(n > 0) {

        fmt.Println("Positive");

    } else if(n < 0) {

        fmt.Println("Negative");

    } else {

        fmt.Println("Zero");

    }

}
```

2. Write a go program to print numbers from 1 to 10 using for loop.

```
package main
import "fmt"
func main() {
    for i := 1 ; i <= 10 ; i++ {
        fmt.Println(i);
    }
}
```

3. Write a go program that finds the largest of three numbers using if...else.

```
package main
import "fmt"
func main() {
    var n1, n2, n3 int;

    fmt.Print("Enter n1 : ");
    fmt.Scanf("%d",&n1);

    fmt.Print("Enter n2 : ");
    fmt.Scanf("%d",&n2);

    fmt.Print("Enter n3 : ");
    fmt.Scanf("%d",&n3);

    if(n1 > n2) {
        if(n1 > n3) {
            fmt.Printf("%d is Largest",n1);
        } else {
            fmt.Printf("%d is Largest",n3);
        }
    } else if (n2 > n3) {
        fmt.Printf("%d is Largest",n2);
    } else {
        fmt.Printf("%d is Largest",n3);
    }
}
```

4. Write a go program that prints the multiplication table of a number up to 10 using a for loop.

```
package main
import "fmt"
func main() {

    var n int;

    fmt.Print("Enter a number : ");
    fmt.Scanf("%d",&n);

    for i := 1 ; i <= 10 ; i++ {

        fmt.Printf("%d x %d = %d \n", n, i, n*i)

    }

}
```

1. Write a go program to check if a number is divisible by both 3 and 5

```
package main
import "fmt"
func main() {
    var n int;
    fmt.Print("Enter a number : ");
    fmt.Scanf("%d",&n);

    if(n % 3 == 0) {
        if (n % 5 == 0) {
            fmt.Print("The number is divisible by both 3 and 5\n")
        } else {
            fmt.Print("The number is only divisible by 3\n");
        }
    } else if(n % 5 == 0) {
        fmt.Print("The number is only divisible by 5\n");
    } else {
        fmt.Print("The number is not divisible by 3 or 5\n");
    }
}
```

2. Write a go program that prints the even numbers between 1 and 20 using a for loop

```
package main
import "fmt"
func main() {
    for i:= 2 ; i < 20 ; i++ {

        if(i % 2 == 0) {
            fmt.Println(i);
        }
    }
}
```

3. Write a go program that accepts a number from the user and checks if it is prime or not.

```
package main
import "fmt"
func main() {
    var n, cnt int;
    cnt = 0;

    fmt.Print("Enter a number : ");
    fmt.Scanf("%d",&n);
    for i:= 2 ; i < n ; i++ {
        if(n % i == 0) {
            cnt++;
        }
    }
    if(cnt == 0) {
        fmt.Println("Prime\n");
    } else {
        fmt.Println("Not Prime\n");
    }
}
```

4. Write a go program that uses switch...case to print the name of the day for a given day number (1 for sunday, 2 for monday, etc)

```
package main
import "fmt"
func main() {

    var n int;
    fmt.Print("Enter day number : ");
    fmt.Scanf("%d",&n);
    switch n {
        case 1:
            fmt.Println("Sunday\n");
        case 2:
            fmt.Println("Monday\n");
        case 3:
            fmt.Println("Tuesday\n");
        case 4:
            fmt.Println("Wednesday\n");
        case 5:
            fmt.Println("Thursday\n");
        case 6:
            fmt.Println("Friday\n");
        case 7:
            fmt.Println("Saturday\n");
        default:
            fmt.Println("Invalid Day Number\n");
    }
}
```

1. Write a function to calculate the sum of the first n natural numbers.

```
package main
import "fmt"

func sum(n int) int {
    var s int = 0
    for i := 1 ; i <= n ; i++ {
        s = s + i
    }
    return s
}

func main() {
    var n int

    fmt.Print("Enter a number : ")
    fmt.Scanf("%d",&n)

    fmt.Println("The sum of numbers from 1 to", n , "is", sum(n))
}
```

2. Write a function to find the maximum of three numbers using functions.

```
package main
import "fmt"

func largest(n1, n2, n3 int) {
    if(n1 >= n2 && n1 >= n3) {
        fmt.Println(n1, "is largest");
    } else if (n2 >= n1 && n2 >= n3) {
        fmt.Println(n2, "is largest");
    } else {
        fmt.Println(n3, "is largest");
    }
}

func main() {
    var n1, n2, n3 int;

    fmt.Print("Enter n1 : ");
    fmt.Scanf("%d",&n1);

    fmt.Print("Enter n2 : ");
    fmt.Scanf("%d",&n2);

    fmt.Print("Enter n3 : ");
    fmt.Scanf("%d",&n3);

    largest(n1, n2, n3)
}
```

3. Write a function to reverse a number (e.g., 123 becomes 321)

```
package main
import "fmt"

func reverse(n int) {
    var rem, sum int
    sum = 0

    for n > 0 {
        rem = n % 10
        sum = sum * 10 + rem
        n = n / 10
    }
    fmt.Print("Reverse : ",sum)
}

func main() {
    var n int
    fmt.Print("Number : ")
    fmt.Scanf("%d",&n)
    reverse(n)
}
```

4. Write a function that checks whether a number is even or odd

```
package main
import "fmt"
```

```

func even_odd(n int) {
    if(n % 2 == 0) {
        fmt.Println("Even")
    } else {
        fmt.Println("Odd")
    }
}

```

```

func main() {

    var n int

    fmt.Print("Enter a number : ")
    fmt.Scanf("%d",&n)

    even_odd(n)
}

```

1. Write a function that accepts a string and returns the count of vowels in it.

```

package main
import "fmt"
func countVowels(s string) int
{
    count := 0
    for _, ch := range s
    {
        switch ch
        {
            case 'a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U':
                count++
        }
    }
    return count
}
func main()
{
    var str string
    fmt.Print("Enter a string: ")
    fmt.Scanln(&str)
    fmt.Println("Number of vowels:", countVowels(str))
}

```

2. Write a recursive function to calculate the Fibonacci sequence up to n terms.

```

package main
import "fmt"

// Recursive function to find Fibonacci number
func fibonacci(n int) int
{
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}
func main() {
    var n int

    fmt.Print("Enter number of terms: ")
    fmt.Scan(&n)
    fmt.Println("Fibonacci sequence:")
    for i := 0; i < n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}

```

3. Write a function to find the factorial of a given number using recursion.

```

package main
import "fmt"
func factorial(n int) int {
    if n == 0 || n == 1 {
        return 1
    }
    return n * factorial(n-1)
}

```

```

func main() {
    var num int

    fmt.Print("Enter a number: ")
    fmt.Scan(&num)

    fmt.Println("Factorial =", factorial(num))
}

```

4. Write a function that checks whether a number is prime or not.

```

package main
import "fmt"
func isPrime(n int) bool {
    if n <= 1 {
        return false
    }

    for i := 2; i*i <= n; i++ {
        if n%i == 0 {
            return false
        }
    }
    return true
}
func main() {
    num := 17
    if isPrime(num) {
        fmt.Println(num, "is a prime number")
    } else {
        fmt.Println(num, "is not a prime number")
    }
}

```

1. Write a go program to store the names and ages of 5 students using arrays. Then print them.

```

package main
import "fmt"
func main() {
    var names[5] string;
    var ages[5] int;

    for i := 0 ; i < 5 ; i++ {
        fmt.Print("Enter name of student ", i + 1 , " : ");
        fmt.Scan(&names[i]);

        fmt.Print("Enter age of student ", i + 1 , " : ");
        fmt.Scan(&ages[i]);
    }
    fmt.Println();
    for i := 0 ; i < 5 ; i++ {
        fmt.Println("Student " , i + 1 , " name : " , names[i]);
        fmt.Println("Student " , i + 1 , " age : " , ages[i]);
        fmt.Println();
    }
}

```

2. Create a program to find the sum and average of 10 numbers using an array.

```

package main
import "fmt"
func main() {
    var numbers[10] int;
    var sum int = 0;
    var avg float64 = 0;

    for i:=0 ; i < 10 ; i++ {
        fmt.Print("Number " , i + 1 , " : ");
        fmt.Scan(&numbers[i]);
    }
    for i:= 0 ; i < 10 ; i++ {
        sum = sum + numbers[i];
    }

    avg = float64(sum) / 10;
}

```

```

        fmt.Println();
        fmt.Println("Sum : " , sum);
        fmt.Println("Average : " , avg);
        fmt.Println();
    }

3. Implement a program to reverse an array of integers.
package main
import "fmt"
func main() {

```

```

    var numbers[5] int;
    var rev[5] int;

    fmt.Println("Enter 5 elements of array \n");
    for i := 0 ; i < 5 ; i++ {

        fmt.Scan(&numbers[i]);

    }
    for i := 0 ; i < 5 ; i++ {

        rev[i] = numbers[5 - 1 - i];

    }
    fmt.Println();
    fmt.Print("Original Array : ");

    for i := 0 ; i < 5 ; i++ {

        fmt.Print(numbers[i] , " ");

    }
    fmt.Println();
    fmt.Print("Reversed Array : ");

    for i := 0 ; i < 5 ; i++ {

        fmt.Print(rev[i] , " ");

    }
    fmt.Println();
}

```

1. Write a go program to create a slice of integers and append 5 random numbers to it. Print the updated slice.

```

package main
import (
    "fmt"
    "math/rand"
)
func main() {

    var numbers[] int;

    fmt.Println("\nAppending 5 random integers \n");

    for i := 0 ; i < 5 ; i++ {

        rn := rand.Intn(100);
        numbers = append(numbers, rn);

    }
    fmt.Print("Integer Slice :", numbers);
    fmt.Println();
}

```

2. Implement a program to slice a slice and print a sub-slice containing elements from index 2 to 4.

```

package main
import "fmt"
func main() {

    var MainSlice[] int;
    var SubSlice[] int;

```



```

        MainSlice = make([]int, 5);
        fmt.Println("Insert 5 elements in Slice\n");

        for i := 0 ; i < 5 ; i++ {
            fmt.Scan(&MainSlice[i]);
        }
        SubSlice = MainSlice[2:5]
        fmt.Println("\nSub Slice :", SubSlice);
    }

```

3. Create a slice of strings and sort them alphabetically.

```

package main
import (
    "fmt"
    "sort"
)
func main() {

    var slice[] string;

    slice = make([]string, 5);
    fmt.Println("Enter 5 strings for slice\n");

    for i := 0 ; i < 5 ; i++ {
        fmt.Scan(&slice[i]);
    }
    sort.Strings(slice)
    fmt.Println("\nSorted Slice :", slice);
}

```

1. Write a Go program to define a Book struct with fields for Title, Author, and Year. Initialize and print the struct's values.

```

package main
import "fmt"
type Book struct {
    Title  string
    Author string
    Year   int
}

func main() {

    b1 := Book{
        Title:  "Go Programming",
        Author: "abc",
        Year:   2023,
    }
    fmt.Println("Title:", b1.Title)
    fmt.Println("Author:", b1.Author)
    fmt.Println("Year:", b1.Year)
}

```

2. Write a Go program to define a Rectangle struct with methods to calculate the Area and Perimeter.

```

package main
import "fmt"
type Rectangle struct {
    Length float64
    Breadth float64
}
func (r Rectangle) Area() float64 {
    return r.Length * r.Breadth
}
func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Length + r.Breadth)
}
func main() {
    rect := Rectangle{Length: 10, Breadth: 5}

    fmt.Println("Area:", rect.Area())
    fmt.Println("Perimeter:", rect.Perimeter())
}

```

3. Write a Go program to create a struct Person with fields Name and Age, then implement a method IsAdult that returns true if the person is 18 or older.

```
package main
import "fmt"
type Person struct {
    Name string
    Age  int
}
func (p Person) IsAdult() bool {
    return p.Age >= 18
}
func main() {
    p1 := Person{Name: "Ali", Age: 20}

    fmt.Println("Name:", p1.Name)
    fmt.Println("Age:", p1.Age)
    fmt.Println("Is Adult:", p1.IsAdult())
}
```

4. Write a Go program to create a Student struct with Name and RollNo fields. Create a method DisplayInfo to print out the details of the student.

```
package main
import "fmt"
type Student struct {
    Name    string
    RollNo  int
}
func (s Student) DisplayInfo() {
    fmt.Println("Student Name:", s.Name)
    fmt.Println("Roll No:", s.RollNo)
}
func main() {
    s1 := Student{Name: "Rahul", RollNo: 101}
    s1.DisplayInfo()
}
```

1. Create a Circle struct with fields for radius. Write methods for Area and Circumference. Call the methods for a circle with a radius of 5.

```
package main
import (
    "fmt"
    "math"
)
type Circle struct {
    radius float64
}
func (c Circle) Area() {
    fmt.Println("The area of circle is " , (math.Pi * math.Pow(c.radius, 2)))
}
func (c Circle) Circumference() {
    fmt.Println("The circumference of circle is " , (2 * math.Pi * c.radius))
}
func main() {
    c := Circle{5}
    c.Area()
    c.Circumference()
}
```

2. Write a Go program that defines a Book struct with Title, Author, and Price fields. Create a method to apply a discount and return the new price of the book.

```
package main
import "fmt"
type Book struct {
    Title  string
    Author string
    Price  float64
}
func (b Book) ApplyDiscount(discount float64) float64 {
    return b.Price - (b.Price * discount / 100)
}
func main() {
    b1 := Book{
        Title: "Go Programming",
        Author: "John Doe",
        Price: 500,
    }
```

```

    }
    newPrice := b1.ApplyDiscount(10)

    fmt.Println("Original Price:", b1.Price)
    fmt.Println("Discounted Price:", newPrice)
}

```

3. Write a Go program to create a BankAccount struct that includes fields for Balance and AccountHolderName. Implement methods for Deposit and Withdraw to update the balance.

```

package main
import "fmt"
type BankAccount struct {
    AccountHolderName string
    Balance            float64
}
func (b *BankAccount) Deposit(amount float64) {
    b.Balance += amount
    fmt.Println("Deposited:", amount)
}
func (b *BankAccount) Withdraw(amount float64) {
    if amount > b.Balance {
        fmt.Println("Insufficient balance")
        return
    }
    b.Balance -= amount
    fmt.Println("Withdrawn:", amount)
}
func main() {
    acc := BankAccount{
        AccountHolderName: "abc",
        Balance:            1000,
    }
    acc.Deposit(500)
    acc.Withdraw(300)
    fmt.Println("Final Balance:", acc.Balance)
}

```

4. Define a Shape interface with methods Area() and Perimeter(). Implement the interface for both a Rectangle and Circle struct.

```

package main
import (
    "fmt"
    "math"
)
type Shape interface {
    Area() float64
    Perimeter() float64
}
type Rectangle struct {
    length float64
    width float64
}
func (r Rectangle) Area() float64 {
    return r.length * r.width
}
func (r Rectangle) Perimeter() float64 {
    return 2 * (r.length + r.width)
}
type Circle struct {
    radius float64
}
func (c Circle) Area() float64 {
    return math.Pi * c.radius * c.radius
}
func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.radius
}
func main() {
    c := Circle{radius : 5}
    r := Rectangle{length : 5, width : 10}
    var s Shape
    s = r
    fmt.Println("Rectangle Area : ", s.Area())
    fmt.Println("Rectangle Perimeter : ", s.Perimeter())
    s = c
}

```

```

        fmt.Println("Circle Area : ", s.Area())
        fmt.Println("Circle Perimeter : ", s.Perimeter())
    }

```

1. Write a Go program that demonstrates the basic concept of a goroutine. (The program should print a simple message in the background using go keyword. Show how the main function waits for completion before exiting.)

```

package main
import (
    "fmt"
    "time"
)
func printMessage() {
    fmt.Println("Hello from goroutine!")
}
func main() {
    go printMessage() // start goroutine

    time.Sleep(1 * time.Second) // wait for goroutine to finish
    fmt.Println("Main function ends")
}

```

2. Write a Go program using a goroutine to print numbers 1 through 5 with a delay of 1 second between each print.

```

package main
import (
    "fmt"
    "time"
)
func printNumbers() {
    for i := 1; i <= 5; i++ {
        fmt.Println(i)
        time.Sleep(1 * time.Second)
    }
}
func main() {
    go printNumbers()
    time.Sleep(6 * time.Second) // wait for completion
}

```

3. Write a Go program that demonstrates launching multiple goroutines. (Each goroutine should print the index of a loop (0 to 4) with a delay of 500ms.)

```

package main
import (
    "fmt"
    "time"
)
func printIndex(i int) {
    time.Sleep(500 * time.Millisecond)
    fmt.Println("Index:", i)
}
func main() {
    for i := 0; i < 5; i++ {
        go printIndex(i)
    }
    time.Sleep(2 * time.Second) // wait for all goroutines
}

```

4. Create a Go program that runs two goroutines concurrently to calculate the square and the cube of a number (e.g., 5).

```

package main
import (
    "fmt"
    "time"
)
func square(n int) {
    fmt.Println("Square:", n*n)
}
func cube(n int) {
    fmt.Println("Cube:", n*n*n)
}
func main() {
    num := 5
    go square(num)
    go cube(num)
    time.Sleep(1 * time.Second) // wait for both
}

```

5. Write a Go program to simulate a long-running task using a goroutine. (The task should run for 10 seconds, and the main function should wait until the task is done using channels or `sync.WaitGroup`.)

```
package main
import (
    "fmt"
    "sync"
    "time"
)
func longTask(wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Println("Task started...")
    time.Sleep(10 * time.Second)
    fmt.Println("Task completed!")
}
func main() {
    var wg sync.WaitGroup
    wg.Add(1)
    go longTask(&wg)
    wg.Wait() // wait for goroutine
    fmt.Println("Main function exits")
}
```

1. Write a Go program that demonstrates how to use `sync.Mutex` to synchronize access to shared data. (Create a program where two goroutines modify a shared integer and print the result.)

```
package main
import (
    "fmt"
    "sync"
)
var count = 0
var mu sync.Mutex
func increment() {
    mu.Lock()
    count++
    fmt.Println("Count:", count)
    mu.Unlock()
}
func main() {
    var wg sync.WaitGroup
    wg.Add(2)
    go func() {
        defer wg.Done()
        increment()
    }()
    go func() {
        defer wg.Done()
        increment()
    }()
    wg.Wait()
}
```

2. Create a Go program that uses `sync.WaitGroup` to synchronize the completion of multiple goroutines. (Launch three goroutines that perform tasks and then print a final message after all tasks are completed.)

```
package main
import (
    "fmt"
    "sync"
    "time"
)
func task(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Println("Task", id, "started")
    time.Sleep(1 * time.Second)
    fmt.Println("Task", id, "completed")
}
func main() {
    var wg sync.WaitGroup
    for i := 1; i <= 3; i++ {
        wg.Add(1)
        go task(i, &wg)
    }
    wg.Wait()
    fmt.Println("All tasks completed")
}
```

3. Write a Go program that uses a `sync.Mutex` to avoid race conditions in a bank account balance update. (Simulate multiple goroutines depositing money into an account.)

```
package main
import (
    "fmt"
    "sync"
)
var balance = 0
var mu sync.Mutex
func deposit(amount int, wg *sync.WaitGroup) {
    defer wg.Done()
    mu.Lock()
    balance += amount
    mu.Unlock()
}
func main() {
    var wg sync.WaitGroup

    for i := 0; i < 5; i++ {
        wg.Add(1)
        go deposit(100, &wg)
    }
    wg.Wait()
    fmt.Println("Final Balance:", balance)
}
```

4. Write a Go program where two goroutines increment a shared counter variable. (Use `sync.Mutex` to ensure the counter is updated safely.)

```
package main
import (
    "fmt"
    "sync"
)
var counter = 0
var mu sync.Mutex
func increment(wg *sync.WaitGroup) {
    defer wg.Done()

    for i := 0; i < 5; i++ {
        mu.Lock()
        counter++
        mu.Unlock()
    }
}
func main() {
    var wg sync.WaitGroup

    wg.Add(2)

    go increment(&wg)
    go increment(&wg)

    wg.Wait()
    fmt.Println("Final Counter:", counter)
}
```

5. Create a Go program that demonstrates the use of `sync.WaitGroup` and `sync.Mutex` together. (Ensure the correct synchronization when multiple goroutines are working on shared resources.)

```
package main
import (
    "fmt"
    "sync"
)
var total = 0
var mu sync.Mutex
func add(value int, wg *sync.WaitGroup) {
    defer wg.Done()

    mu.Lock()
    total += value
    mu.Unlock()
}
func main() {
    var wg sync.WaitGroup
    for i := 1; i <= 5; i++ {
        wg.Add(1)
```

```

        go add(i, &wg)
    }

    wg.Wait()
    fmt.Println("Total Sum:", total)
}

```

1. Write a Go program to declare and initialize three variables : an integer, a float, and a string. Then, print the sum of the integer and float values.

```

package main
import "fmt"
func main() {
    var n1 int = 10
    var n2 float32 = 15.5
    //var str string = "Syed"
    n3 := float32(n1) + n2

    fmt.Print("Integer + Float : ",n3)
}

```

2. Write a go pogram to convert a given temperature from Celsius to Fahrenheit (formula: $F = C * 9 / 5 + 32$). Take the temperature in Celsius as input and display the result in Fahrenheit.

```

package main
import "fmt"

func main() {
    var C float64;

    fmt.Print("Enter temperature in Celsius : ")
    fmt.Scanf("%f", &C);

    var F float64 = C * 9.0 / 5.0 + 32;

    fmt.Println("The temperature in Fahrenheit : ", F);
}

```

3. Write a go program to find the area of a trianble. Take the base and height as input and display the area.

```

package main
import "fmt"
func main() {
    var base, height, area float64;

    fmt.Print("Enter height of triangle : ");
    fmt.Scanf("%f", &height);

    fmt.Print("Enter base of triangle : ");
    fmt.Scanf("%f", &base);

    area = (base * height) / 2;

    fmt.Println("The area of triangle is : ", area);
}

```

4. Write a go program to calculate the simple interest. Take principle amount, rate of interest, and time period as inputs and display the interest.

```

package main
import "fmt"
func main() {

    var P, R, T, SI float64;

    fmt.Print("Enter Principle Amount : ");
    fmt.Scanf("%f",&P);

    fmt.Print("Enter Rate of Interest : ");
    fmt.Scanf("%f",&R);

    fmt.Print("Enter Time Period : ");
    fmt.Scanf("%f",&T);

    SI = (P * R * T) / 100;

    fmt.Println("The Calculated Simple Interest : ", SI);
}

```

1. Write a Go program that prints the sum of numbers from 1 to n, where n is given by the user.

```
package main
import "fmt"
func main() {

    var n, sum int;
    sum = 0;
    fmt.Print("Enter a number : ");
    fmt.Scanf("%d",&n);

    for i:= 1 ; i <= n ; i++ {
        sum = sum + i;
    }
    fmt.Println("The sum from 1 to n : ",sum);
}
```

2. Write a Go program that prints the Fibonacci series up to n terms.

```
package main
import "fmt"
func main() {
    var n int

    fmt.Print("Enter the number of terms: ")
    fmt.Scan(&n)

    a, b := 0, 1

    fmt.Println("Fibonacci Series up to", n, "terms:")
    for i := 0; i < n; i++ {
        fmt.Print(a, " ")
        a, b = b, a+b
    }
}
```

3. Write a Go program that uses a for loop to find the factorial of a number.

```
package main
import "fmt"
func main() {

    var n, fact int

    fact = 1;

    fmt.Print("Enter the number : ")
    fmt.Scan(&n)

    for i:= 1 ; i <= n ; i++ {
        fact = fact * i;
    }
    fmt.Println("Factorial : ",fact)
}
```

4. Write a Go program to check if a number is a perfect number (a perfect number is a number that is the sum of its proper divisors).

```
package main
import "fmt"
func main() {

    var n, sum int
    sum = 0;

    fmt.Print("Enter number : ");
    fmt.Scanf("%d",&n);

    for i:=1 ; i < n ; i++ {
        if(n % i == 0) {
            sum = sum + i;
        }
    }
    if(sum == n) {
        fmt.Println("Perfect Number");
    } else {
        fmt.Println("Not Perfect Number")
    }
}
```



```

}
1. Write a function that accepts a number and returns both its square and cube.
package main
import "fmt"

```

```

func squareAndCube(n int) (int, int) {
    return n*n, n*n*n
}
func main() {
    var num int
    fmt.Print("Enter a number: ")
    fmt.Scan(&num)

    sq, cube := squareAndCube(num)
    fmt.Println("Square:", sq)
    fmt.Println("Cube:", cube)
}

```

2. Write a function that accepts two numbers and returns their product and quotient.

```

package main
import "fmt"

func calculate(a, b float64) (float64, float64) {
    product := a * b
    quotient := a / b
    return product, quotient
}
func main() {
    var num1, num2 float64

    fmt.Print("Enter first number: ")
    fmt.Scan(&num1)

    fmt.Print("Enter second number: ")
    fmt.Scan(&num2)

    product, quotient := calculate(num1, num2)

    fmt.Println("Product:", product)
    fmt.Println("Quotient:", quotient)
}

```

3. Write a function that returns the greatest common divisor (GCD) of two numbers.

```

package main
import "fmt"
func gcd(a, b int) int {
    for b != 0 {
        a, b = b, a%b
    }
    return a
}
func main() {
    var x, y int

    fmt.Print("Enter two numbers: ")
    fmt.Scan(&x, &y)

    fmt.Println("GCD:", gcd(x, y))
}

```

4. Write a function that finds the sum of all the digits of a number.

```

package main
import "fmt"
func main() {
    var num, sum int

    fmt.Print("Enter a number: ")
    fmt.Scan(&num)

    for num != 0 {
        sum += num % 10
        num = num / 10
    }
    fmt.Println("Sum of digits:", sum)
}

```

```
}
1. Create a map to store the student name as the key and their marks as the value. Insert 5
student's data and print the map.
```

```
package main
import "fmt"
func main() {

    students := make(map[string]int)

    for i := 1 ; i <= 5 ; i++ {

        var name string;
        var marks int;

        fmt.Print("Student ", i , " name : ");
        fmt.Scan(&name);
        fmt.Print("Student ", i , " marks : ");
        fmt.Scan(&marks);

        students[name] = marks;

        fmt.Println();

    }

    fmt.Println("Data of 5 Students\n\n");
    for name, marks := range students {
        fmt.Println(name, ":", marks);
    }

}
```

2. Write a go program to find the maximum value from a slice of integers.

```
package main
import "fmt"
func main() {

    var numbers[] int;

    numbers = make([]int, 5);

    fmt.Println("Enter 5 numbers\n");

    for i := 0 ; i < 5 ; i++ {

        fmt.Scan(&numbers[i]);

    }
    max := numbers[0];
    for i := range numbers {

        if(numbers[i] > max) {
            max = numbers[i];
        }

    }
    fmt.Println("\nMaximum : ", max);

}
```

3. Create a program that accepts user input and stores it in a map with the product name as the key and its price as the value. Print the map

```
package main
import "fmt"
func main() {

    products := make(map[string]int)

    var name string;
    var price int;

    fmt.Print("Product Name : ");
    fmt.Scan(&name);
    fmt.Print("Product Price : ");
    fmt.Scan(&price);

    products[name] = price;
    fmt.Println("\n",products);

}
```

}
1. Create a Book struct with Title and Author fields. Define a method GetDetails that returns the title and author of the book in a formatted string.

```
package main
import "fmt"
type Book struct {

    Title string
    Author string
}
func (b Book) GetDetails() string {
    return fmt.Sprintf("Title : %s \nAuthor : %s", b.Title, b.Author)
}
func main() {
    b := Book{"Harry Potter", "J K Rowling"}
    fmt.Println(b.GetDetails());
}
```

2. Write a Go program to define a Person struct with fields Name and Age. Implement a method Greet that prints a greeting using the person's name.

```
package main
import "fmt"
type Person struct {
    Name string
    Age int
}
func (p Person) Greet() {
    fmt.Println("Hello!",p.Name)
}
func main() {
    p := Person{"abc",19}
    p.Greet()
}
```

3. Write a Go program to define a Movie struct with Title, Director, and Year fields. Implement a method DisplayDetails to print the details of the movie.

```
package main
import "fmt"
type Movie struct {
    Title string
    Director string
    Year int
}
func (m Movie) DisplayDetails() {
    fmt.Println("Title :", m.Title);
    fmt.Println("Director :", m.Director);
    fmt.Println("Year :", m.Year);
}
func main() {
    m := Movie{"Harry Potter", "J K Rowling", 2000}
    m.DisplayDetails()
}
```

4. Define an interface Animal with a method Speak(). Implement this interface with Dog and Cat structs, and call their Speak() method to output sound.

```
package main
import "fmt"
type Animal interface {
    Speak()
}
type Dog struct {}
type Cat struct {}
func (d Dog) Speak() {
    fmt.Println("Dog Barks")
}
func (c Cat) Speak() {
    fmt.Println("Cat Meows")
}
func main() {
    var a Animal
    a = Dog{}
    a.Speak()
    a = Cat{}
    a.Speak()
}
```

```

}
1. Write a Go program that demonstrates sending and receiving data from a channel. (The
program should send an integer value from a goroutine to the main function and print it.)
package main
import "fmt"
func sendData(ch chan int) {
    ch <- 10 // send value
}
func main() {
    ch := make(chan int)

    go sendData(ch)

    value := <-ch // receive value
    fmt.Println("Received:", value)
}

```

2. Create a Go program that demonstrates the use of a buffered channel. (Send and receive multiple values through a channel with a buffer size of 3 and print the values in the main function.)

```

package main
import "fmt"
func main() {
    ch := make(chan int, 3) // buffered channel
    ch <- 1
    ch <- 2
    ch <- 3

    fmt.Println(<-ch)
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}

```

3. Write a Go program that uses a channel to synchronize two goroutines. (One goroutine should send a value to the channel, and the other should receive it and print it.)

```

package main
import "fmt"
func sender(ch chan string) {
    ch <- "Hello from sender"
}
func receiver(ch chan string) {
    msg := <-ch
    fmt.Println(msg)
}
func main() {
    ch := make(chan string)

    go sender(ch)
    go receiver(ch)

    // wait so goroutines finish
    var input string
    fmt.Scanln(&input)
}

```

4. Create a Go program to simulate a ping-pong game between two goroutines using channels. (One goroutine sends a message ("Ping") to another, which responds with ("Pong").)

```

package main
import "fmt"
func ping(ch chan string) {
    ch <- "Ping"
}
func pong(ch chan string) {
    msg := <-ch
    fmt.Println(msg)
    ch <- "Pong"
}
func main() {
    ch := make(chan string)
    go ping(ch)
    go pong(ch)
    fmt.Println(<-ch)
}

```

5. Write a Go program that demonstrates closing a channel. (After sending some values through the channel, close the channel, and show how receiving from a closed channel works.)

```
package main

import "fmt"

func main() {
    ch := make(chan int)

    go func() {
        ch <- 1
        ch <- 2
        ch <- 3
        close(ch) // close channel
    }()

    for value := range ch { // receive until closed
        fmt.Println(value)
    }

    fmt.Println("Channel closed")
}
```